GitHub Customer Churn Predictor - Report

Final project report for a Dockerized FastAPI machine learning application using public GitHub activity data.

GitHub Repository: https://github.com/JoseTorresHD/churn-predictor

1. Project Overview

This project implements a containerized machine learning API to predict customer churn based on public GitHub user activity. The application collects user profile data from the GitHub REST API, generates behavioral and profile-based features, applies multiple feature selection methods, trains a classification model, and exposes the prediction model through a FastAPI service.

The application is packaged with Docker and can be executed using Docker Compose. The project is self-contained: it includes the trained model, raw data, generated features, model metrics, a notebook, README documentation, and this written report.

2. Data Source

The data source used in this project is the public GitHub REST API. User profiles were collected from GitHub users and organizations discovered through popular repositories and seed queries. The collected raw dataset contains 250 GitHub profiles.

The raw data is stored in: data/raw/github_users.csv

A GitHub Personal Access Token can be used through the GITHUB_TOKEN environment variable to avoid unauthenticated API rate limits. The token is not stored in the source code.

3. Churn Definition

For this project, churn is defined as inactivity based on the user last profile update date. A user is labeled as churned when: days_since_last_activity > 90.

This means that if a GitHub profile has not shown activity or updates in more than 90 days, it is considered churned for the purpose of this model.

The generated class balance was: Active users: 157; Churned users: 93. This gives a reasonably balanced dataset for a small academic churn prediction project.

4. Feature Engineering

The project generates 14 features from the raw GitHub profile data. These features combine activity recency, account maturity, repository behavior, follower/following relationships, profile completeness, and account type.

Time-based features: account_age_days, days_since_last_activity. Ratio features: follower_ratio, followers_per_repo, following_per_repo. Aggregation features: repos_per_year, gists_per_year, engagement_score. Binary features: has_no_repos, has_company, has_blog, has_location, is_hireable, is_organization.

5. Exploratory Data Analysis

The EDA notebook is located at notebooks/eda_and_selection.ipynb. It documents the raw dataset loading, feature generation, churn label distribution, descriptive feature statistics, variance threshold analysis, correlation matrix, feature selection comparison, model comparison, and final conclusions.

The notebook acts as an audit trail for the data science workflow and supports reproducibility.

6. Feature Selection

Four feature selection methods were applied: 1) Filter method using ANOVA F-test, 2) Recursive Feature Elimination using Logistic Regression, 3) Decision Tree feature importance, and 4) Random Forest feature importance.

The notebook also includes filter-based variance threshold and correlation matrix analysis. The comparison table is saved in data/raw/feature_selection_comparison.csv.

The strongest selected features included days_since_last_activity, is_organization, following_per_repo, and is_hireable. Other optional features included has_company, follower_ratio, engagement_score, has_blog, has_location, and gists_per_year.

7. Model Training

The final production model uses a Random Forest classifier and is saved as app/model.pkl. The API currently uses the following operational features: days_since_last_activity, repos_per_year, follower_ratio, followers_per_repo, and engagement_score.

Training metrics are saved in data/raw/model_metrics.csv. The trained model is included in the repository so that the API can run immediately after cloning and building the Docker image.

8. Model Comparison

Because the churn label is directly related to inactivity, a second model variant was trained without the days_since_last_activity feature. This was done to avoid relying only on the same variable used to define churn and to evaluate whether indirect behavioral signals could also predict churn.

The comparison was saved in data/raw/model_comparison.csv.

Model	Accuracy	Precision	Recall	F1
With recency	1.00	1.00	1.00	1.00
Without days_since_last_activity	0.60	0.46	0.57	0.51

The model with recency performs perfectly because days_since_last_activity is strongly tied to the churn definition. The model without recency has lower but more realistic performance, showing that other signals such as follower behavior, organization status, company information, and engagement can still provide useful predictive value.

9. API Endpoints

The FastAPI application exposes the following endpoints:

GET /health - Returns the health status of the API.

GET /features - Returns the list of features expected by the prediction endpoint.

POST /predict - Receives a JSON payload with required features and returns a churn prediction and probability score.

Example payload:

```
{"days_since_last_activity":120,"repos_per_year":1.5,"follower_ratio":0.4,"followers_per_repo"
:1.0,"engagement_score":30}
```

Example response:

```
{"churned":true,"churn_probability":0.92}
```

Docker execution command:

```
sudo docker compose up -d --build
```

10. Retention Strategy

- Send reactivation messages to users inactive for more than 90 days.

- Offer personalized recommendations based on repository activity.

- Prioritize users with previously high engagement scores.

- Monitor users with declining repository creation or interaction rates.

- Segment organizations and individual users separately because behavior patterns differ.

11. Ethical Considerations and Limitations

- This project uses public GitHub profile data only. No private repositories, private user information, or sensitive data are collected.

- The dataset is small compared to a production churn system.

- The churn label is inferred from GitHub activity and may not represent actual customer cancellation.

- days_since_last_activity is highly predictive because it is directly related to the churn definition.

- GitHub profile update dates may not capture all forms of developer activity.

- The model should be validated with a larger dataset before real-world use.

Despite these limitations, the project demonstrates a complete machine learning workflow including data collection, feature engineering, feature selection, model training, API deployment, and Docker-based execution.

12. Deliverables

- GitHub repository: https://github.com/JoseTorresHD/churn-predictor

- Dockerfile and docker-compose.yml

- FastAPI application with /health, /features, and /predict endpoints

- Raw and generated data files under data/raw/

- EDA and feature selection notebook

- Trained model saved as app/model.pkl

- README documentation

- Written report in Markdown and PDF formats

Appendix - Reproducibility Commands

A professor or reviewer can clone the repository and execute:

```
git clone https://github.com/JoseTorresHD/churn-predictor.git cd churn-predictor sudo docker compose up -d --build sleep 8 curl http://127.0.0.1:8000/health curl http://127.0.0.1:8000/features curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{"days_since_last_activity":120,"repos_per_year":1.5,"follower_ratio":0.4,"followers_per_repo":1.0,"engagement_score":30}' sudo docker compose down
```

Expected output includes a healthy API response, the list of required features, and a churn prediction similar to:

```
{"churned":true,"churn_probability":0.92}
```